JWT Security Analysis Report

Prepared by: Charmaine Mangorima
Date: November 30, 2025
Tools used: Burpsuite Community Edition, xjwt.io

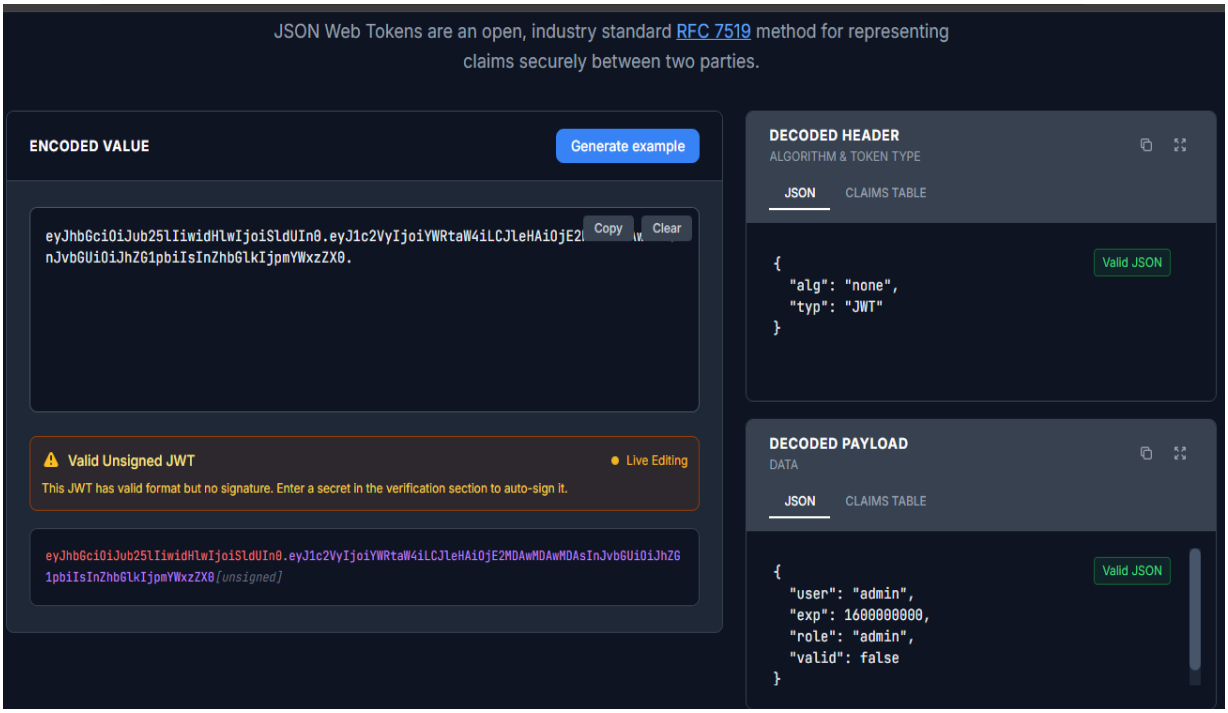
Executive Summary

This report documents the security analysis of a JWT token. The token has multiple critical security vulnerabilities that would allow complete authentication and authorization bypass in any system accepting this token.

5 critical security flaws were identified in the token structure and implementation. These vulnerabilities collectively enable attackers to:

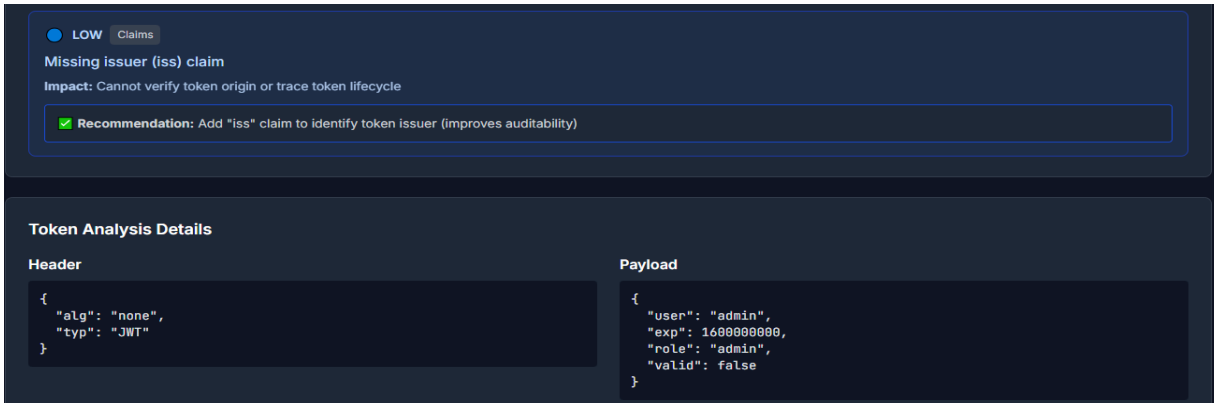
- Forge tokens without cryptographic keys
- Impersonate any user including administrators
- Bypass expiration checks
- Manipulate authorization controls
- Exploit weak validation logic

Critical Finding: This token uses the "none" algorithm meaning that the token is not signed, which is the most severe JWT vulnerability possible and renders all other security controls meaningless. I used xjwt.io's JWT Encoder/Decoder to see the structure of the JWT as shown below:



As shown below, the security flaws are:

- No signature
- Expired token
- Missing timestamp validation(iat,nbf)
- Missing audience claim
- Missing issuer claim



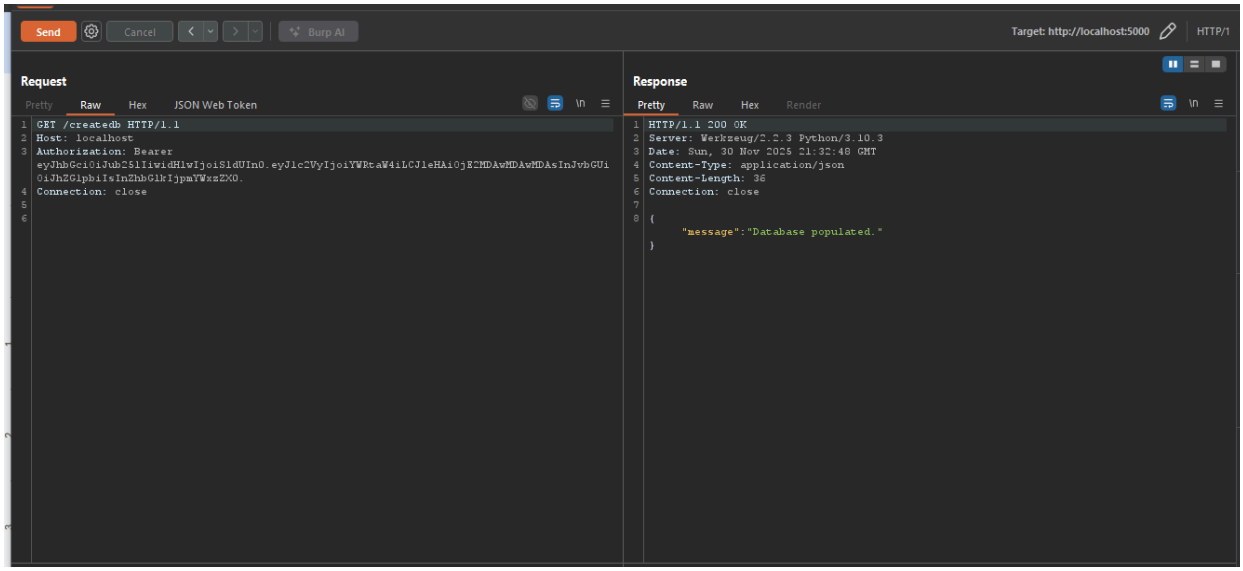
Findings

No signature algorithm - “alg”:”none”

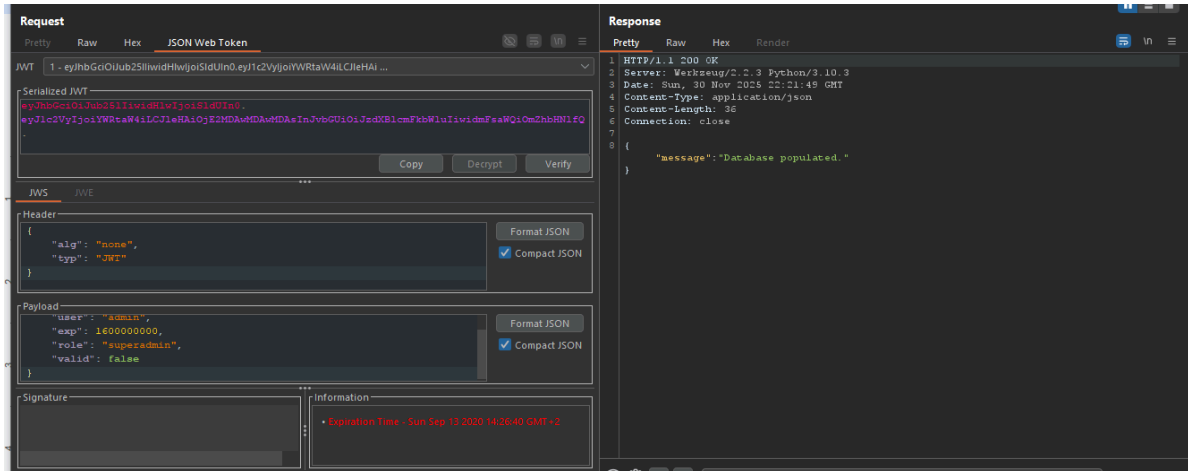
Why this is a vulnerability: The header contains "alg": "none", meaning the token is unsecured and has no cryptographic signature, so the server cannot detect tampering.

How attackers exploit:

In Burpsuite, I first sent a request to my running backend on localhost:5000 on the /createdb endpoint with the vulnerable JWT in the request header. The server accepted this request.



I modified the role to superadmin, which modifies the payload on the token but the request was still accepted by the server. An attacker can escalate privileges and use this token to access unauthorized data.

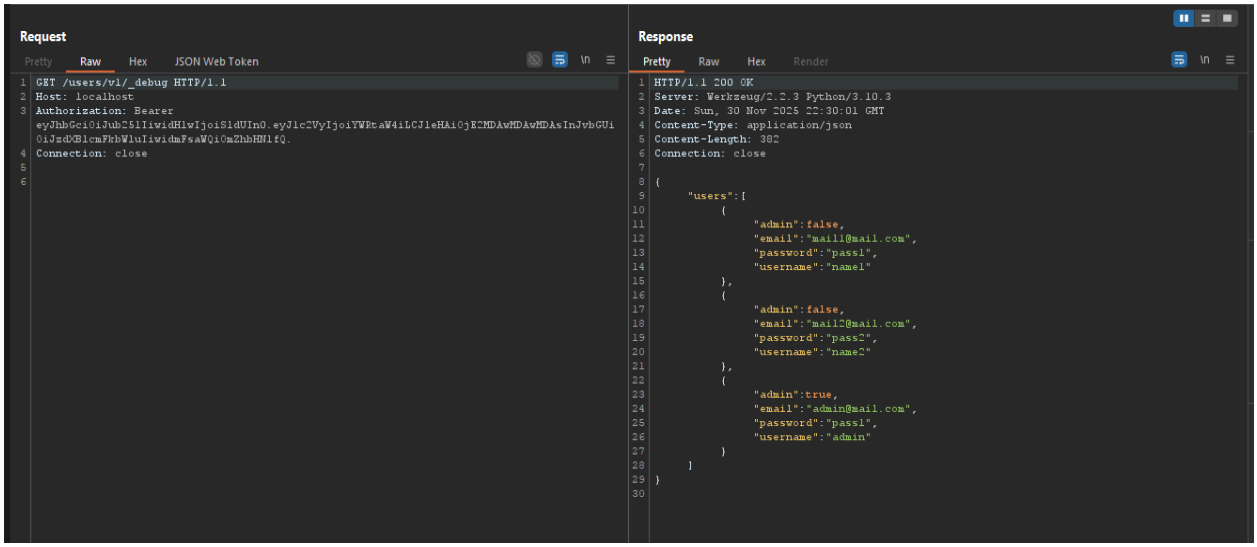


2. Token Has Already Expired (exp long in the past)

As shown above, the token expired on Sunday Sep 13 2020 14:26:40 GMT+2.

Why this is a vulnerability:If a server still accepts an expired token, it is vulnerable to token replay attacks, meaning an attacker can reuse old tokens to regain access forever.

How attackers exploit:
I used the same expired token to access a different endpoint and it returned a response. This is exactly how attackers can use this expired token to replay attacks and access APIs forever.



3. Missing timestamp validation(iat, nbf)

Why this is a vulnerability: The payload lacks iat (issued at) and nbf (not before) claims, so the backend cannot reliably determine when the token was created or from which time it becomes valid, making it harder to detect abuse, clock skew issues or early/late replays.

How attackers exploit: As shown in the examples above, the token is being accepted without checking freshness and that is how attackers use tokens issued before password resets, logout, or privilege changes.

4. Missing audience claim

This means the token can be used across different services.

Why this is a vulnerability: There is no aud (audience) or similar context claim tying the token to a specific service or client, so the same token could be replayed against any system that mistakenly shares the same verification key or trusts the token format

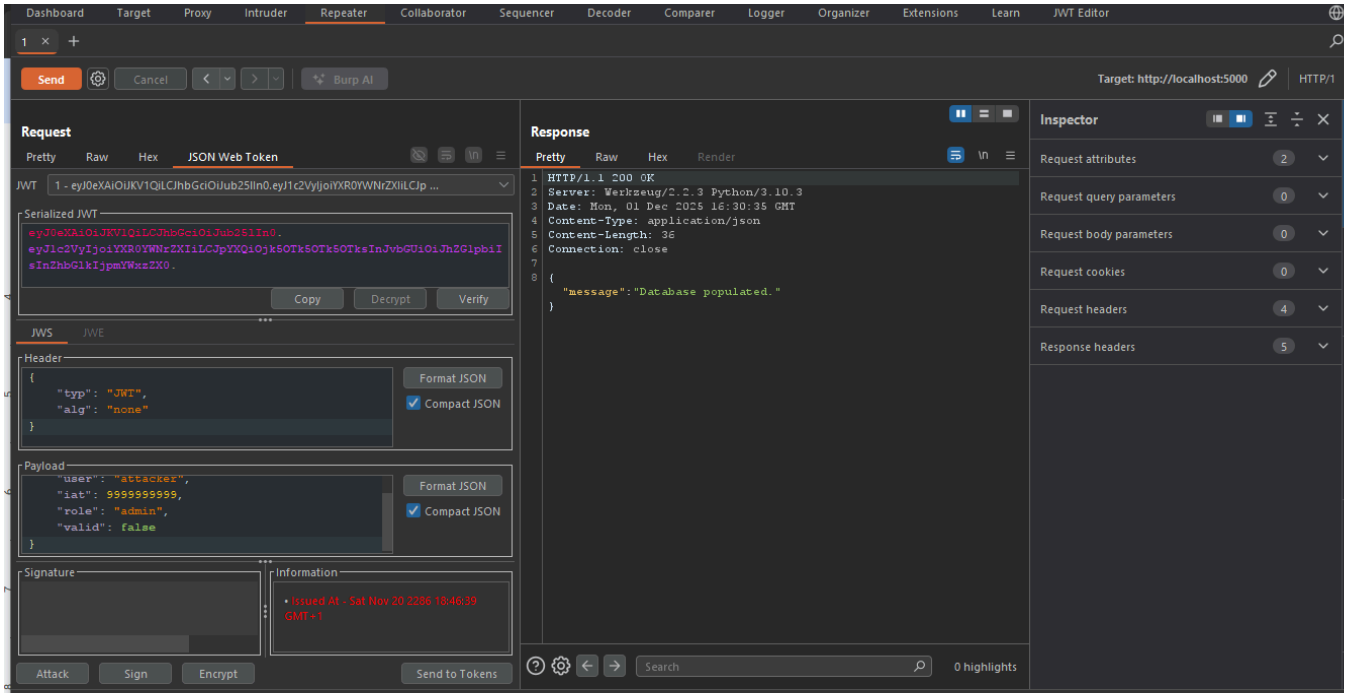
How attackers exploit: An attacker who gets this token from one application could try it against other APIs or environments that accept the same JWTs, achieving cross-service impersonation if any of them do not enforce audience checks.

5. Missing issuer claim

Why this is a vulnerability: A secure JWT should include an iss (issuer) claim that identifies which system generated the token. The server cannot verify which system issued the token. Any attacker can generate a fake token from anywhere and the backend cannot detect it. It destroys trust boundaries in microservices / distributed systems and It allows token confusion attacks (sending a JWT from another application to your API)

How attackers exploit: A missing issuer claim enables the attacker to create their own token and send it to your API, and the API cannot tell if it is fake.

In burpsuit, I modified the payload and set the username to “attacker” but the server accepted it with the role of admin. This means that an attacker can create their own token and access the API.



Conclusion

The analysis of the provided JWT shows that the JWT is critically insecure due to missing validations, lack of a signature and other absent claims required by the industry standard RFC7519 method. Using Burp Suite, it was demonstrated that an attacker can easily forge or modify the token to gain unauthorized access or escalate privileges. These flaws undermine authentication integrity and expose the application to serious security risks. To secure the system, proper JWT best practices such as enforcing signed and stronger algorithms(asymmetric), validating claims like issuer and expiration and rejecting tampered tokens must be implemented.

End of report